TP1 – Le catalogue du disquaire

React Native · TP individuel · Fondations

Un disquaire indépendant veut une application pour présenter ses vinyles. Elle doit fonctionner sur le téléphone des clients et sur l'écran de la boutique, avec le même code. Vous construisez son **catalogue** : une grille d'albums sur grand écran, une liste sur téléphone, une recherche, un filtre par genre et un panier.

Ce TP en bref

- **Objectifs** : créer un projet Expo en TypeScript, modéliser des données, écrire des composants aux props typées, gérer l'état et rendre l'interface responsive.
- **Individuel et non noté**. Il prépare le projet : prenez le temps de comprendre chaque ligne.
- L'IA est autorisée, à une condition : **comprendre chaque ligne** que vous gardez. L'examen écrit se fera sans IA.
- Testez à chaque étape **sur le web et sur votre téléphone** (Expo Go).

0 Le projet et le dépôt

Expo · Git

1. Créez le projet, videz l'exemple fourni puis lancez-le :

   Terminal

   ```
   npx create-expo-app@latest rn-tp1
   cd rn-tp1
   npm run reset-project
   npx expo start
   ```

   À la question de `reset-project`, répondez `n` : l'exemple est supprimé au lieu d'être déplacé.

2. Dans `app.json`, remplacez `"output": "server"` par `"output": "single"` (rubrique `web`), puis relancez avec `npx expo start --clear`.

3. Appuyez sur `w` pour ouvrir la version web, puis scannez le QR code avec Expo Go. Le même écran s'affiche des deux côtés.

4. Sur GitHub, créez un dépôt **privé** nommé `rn-tp1-nom-prenom`, reliez-le à votre projet et poussez un premier commit. Il garde une trace de votre travail : prenez l'habitude de commiter à chaque étape.

Le téléphone ne se connecte pas au projet

Le téléphone et l'ordinateur doivent être sur le même réseau. Si le Wi-Fi de l'école bloque la connexion, lancez `npx expo start --tunnel`.

1 Le modèle et les données

TypeScript · types · unions

Dans le fichier `src/models/album.ts`, écrivez et exportez :

- le type `Genre`, une **union** des valeurs `"rock"`, `"jazz"`, `"pop"`, `"hip-hop"` et `"electro"` ;
- le type `Album`, décrit par le tableau ci-dessous ;
- la constante `GENRES`, un tableau de type `Genre[]` qui contient les cinq genres.

Propriété	Contenu	Type
`id`	identifiant unique	`string`
`title`	titre de l'album	`string`
`artist`	artiste ou groupe	`string`
`year`	année de sortie	`number`
`genre`	genre musical	`Genre`
`price`	prix en euros	`number`
`inStock`	disponible en boutique	`boolean`
`coverUrl`	adresse de la pochette	`string`

Dans le fichier `src/data/albums.ts`, copiez les données du disquaire :

src/data/albums.ts

```ts
import { Album } from "@/models/album";

export const ALBUMS: Album[] = [
  { id: "a1", title: "Kind of Blue", artist: "Miles Davis", year: 1959, genre: "jazz", price: 24.9, inStock: true, coverUrl: "https://picsum.photos/id/100/400/400" },
  { id: "a2", title: "Random Access Memories", artist: "Daft Punk", year: 2013, genre: "electro", price: 34.9, inStock: true, coverUrl: "https://picsum.photos/id/101/400/400" },
  { id: "a3", title: "Rumours", artist: "Fleetwood Mac", year: 1977, genre: "rock", price: 27.5, inStock: false, coverUrl: "https://picsum.photos/id/102/400/400" },
  { id: "a4", title: "To Pimp a Butterfly", artist: "Kendrick Lamar", year: 2015, genre: "hip-hop", price: 32, inStock: true, coverUrl: "https://picsum.photos/id/103/400/400" },
  { id: "a5", title: "Thriller", artist: "Michael Jackson", year: 1982, genre: "pop", price: 22.9, inStock: true, coverUrl: "https://picsum.photos/id/104/400/400" },
  { id: "a6", title: "A Love Supreme", artist: "John Coltrane", year: 1965, genre: "jazz", price: 26, inStock: true, coverUrl: "https://picsum.photos/id/106/400/400" },
  { id: "a7", title: "OK Computer", artist: "Radiohead", year: 1997, genre: "rock", price: 29.9, inStock: true, coverUrl: "https://picsum.photos/id/107/400/400" },
  { id: "a8", title: "Discovery", artist: "Daft Punk", year: 2001, genre: "electro", price: 31.5, inStock: false, coverUrl: "https://picsum.photos/id/108/400/400" },
  { id: "a9", title: "Illmatic", artist: "Nas", year: 1994, genre: "hip-hop", price: 25, inStock: true, coverUrl: "https://picsum.photos/id/109/400/400" },
  { id: "a10", title: "L'Homme à tête de chou", artist: "Serge Gainsbourg", year: 1976, genre: "pop", price: 23.5, inStock: true, coverUrl: "https://picsum.photos/id/110/400/400" },
```

```
  { id: "a11", title: "Moanin'", artist: "Art Blakey", year: 1958, genre: "jazz", price: 21.9, inStock: true, coverUrl: "https://picsum.photos/id/111/400/400" },
  { id: "a12", title: "Homework", artist: "Daft Punk", year: 1997, genre: "electro", price: 28.9, inStock: true, coverUrl: "https://picsum.photos/id/112/400/400" },
];
```

Les pochettes sont des photos d'illustration de picsum.photos.

> **Vérifiez votre type**
>
> Changez temporairement le genre d'un album en `"classique"` : votre éditeur doit souligner l'erreur. Lancez aussi `npx tsc --noEmit` dans le terminal. Il ne doit rien afficher une fois l'erreur corrigée.

2 La carte d'un album

Composants · props typées · Image · StyleSheet

Dans le fichier `src/components/album-card.tsx`, écrivez le composant `AlbumCard`.

- Il reçoit une prop `album` de type `Album`. Déclarez le type de ses props (`AlbumCardProps`).
- Il affiche la pochette carrée, le genre, le titre, « artiste · année » et le prix au format français (`34,90 €`).
- Ses styles sont regroupés dans un `StyleSheet.create`, en bas du fichier.

Indice : afficher un prix en euros

Formatage d'un prix

```
new Intl.NumberFormat("fr-FR", { style: "currency", currency: "EUR" }).for
mat(34.9); // "34,90 €"
```

Indice : la pochette reste invisible

Une image distante n'a pas de taille par défaut. Donnez-lui une largeur et un rapport de proportions : `{ width: "100%", aspectRatio: 1 }`.

Pour tester, affichez la carte du premier album dans `src/app/index.tsx` : `<AlbumCard album={ALBUMS[0]} />`.

3 Un catalogue responsive

FlatList · useWindowDimensions · hooks

1. Dans `src/app/index.tsx`, affichez **tous** les albums avec une `FlatList`.
2. Dans le fichier `src/hooks/use-columns.ts`, écrivez le hook `useColumns()`. Il renvoie le nombre de colonnes selon la largeur de la fenêtre :

Largeur de la fenêtre	Colonnes
moins de 600 px	1
de 600 à 899 px	2
de 900 à 1 199 px	3
1 200 px et plus	4

3. Utilisez ce hook pour passer la liste en grille. Redimensionnez la fenêtre du navigateur : le nombre de colonnes doit changer **sans recharger la page**.
4. Sur un très grand écran, le catalogue ne doit pas dépasser **1 280 px de large** et doit rester centré.

Indice : « Changing numColumns on the fly is not supported »

`FlatList` ne sait pas changer de nombre de colonnes après son premier affichage. Donnez-lui une prop `key` qui dépend du nombre de colonnes : elle sera recréée à chaque changement.

4 Le panier

useState · Pressable · état remonté

1. Ajoutez à chaque carte un bouton **« Ajouter au panier »** (`Pressable`). Un nouvel appui retire l'album du panier.
2. Le bouton donne un **retour visuel** pendant l'appui. Son libellé et sa couleur changent quand l'album est dans le panier (« Dans le panier ✓ »).
3. Un album épuisé (`inStock: false`) affiche un bouton **« Épuisé »** grisé, qui ne réagit pas.
4. En haut de l'écran, affichez le nombre d'albums du panier et leur **prix total** : `Panier : 2 · 59,80 €` .

> **Où placer l'état du panier ?**
>
> La carte affiche l'état du panier, l'en-tête aussi. L'état doit donc vivre dans leur **parent commun**, l'écran. La carte reçoit seulement deux props de plus : `inCart` (un booléen) et `onToggleCart` (une fonction qui reçoit l'identifiant de l'album).

5 Recherche et filtre

TextInput · état dérivé · ScrollView horizontale

1. Ajoutez un champ de recherche (`TextInput`). Il filtre les albums dont le **titre ou l'artiste** contient le texte saisi, sans tenir compte des majuscules.
2. Dans le fichier `src/components/genre-filter.tsx` , écrivez le composant `GenreFilter` . Il affiche une rangée de boutons « Tous », « rock », « jazz »… Le genre choisi est mis en évidence.
3. La recherche et le filtre se **combinent** : « daft » avec le genre « electro » affiche les trois albums de Daft Punk.
4. Quand aucun album ne correspond, la liste affiche un message (prop `ListEmptyComponent`).

Indice : faut-il un état pour la liste filtrée ?

Non. La liste affichée se **calcule** à chaque rendu à partir de `ALBUMS` , du texte recherché et du genre choisi. Seuls ces deux derniers sont des états.

Indice : le type du genre choisi

« Tous » correspond à l'absence de filtre : `useState<Genre | null>(null)` .

+ Pour aller plus loin

- Un tri par prix ou par année, choisi par l'utilisateur.
- Un thème sombre qui suit le réglage du téléphone, avec `useColorScheme()`.
- Un retour haptique (une petite vibration) à l'ajout au panier, avec `expo-haptics`. Sur le web, il ne se passe rien : vérifiez que l'application ne plante pas.
- Sur le web, un curseur en forme de main au survol des boutons.

✓ Vérifier votre travail

- ☐ `npx tsc --noEmit` n'affiche aucune erreur.
- ☐ Le catalogue s'affiche en 1 colonne sur votre téléphone et en 3 ou 4 colonnes sur l'ordinateur.
- ☐ Les types sont dans `src/models/`, les données dans `src/data/`, les composants dans `src/components/`, le hook dans `src/hooks/`.
- ☐ Aucune propriété de style n'est écrite directement dans le JSX : tout passe par `StyleSheet.create`.
- ☐ Le panier, le total, la recherche et le filtre fonctionnent ensemble.
- ☐ Votre dépôt contient un commit par étape.

> **Rien à rendre**
>
> Ce TP sert à vous exercer.